# Assignment 5: Earley Parser Assignment

Gargie Tambe
2022111008

March 27, 2026

## Question 1

**Consider the sentence time flies like an arrow. Emulate the behaviour of the Earley algorithm on this sentence using the grammar rules provided in Figure 1 (ignoring rule probabilities). Provide the final chart. (20 marks)**

### Grammar Used

The grammar from Figure 1 of the assignment (with probabilities) is reproduced below. The grammar is case-sensitive and all rule probabilities for rules sharing the same left-hand side sum to 1.

```
1.0   S    NP VP
0.25  NP   N N
0.40  NP   D N
0.35  NP   N
0.60  VP   V NP
0.40  VP   V ADVP
1.0   ADVP ADV NP
0.40  N    time
0.20  N    flies
0.40  N    arrow
1.0   D    an
1.0   ADV  like
0.50  V    flies
0.50  V    like
```

### Final Earley Chart

The Earley algorithm processes the sentence word by word, maintaining a chart of dotted rules. Each item is of the form $[i, j]$ LHS $\rightarrow \alpha \bullet \beta$ $(w)$, where $[i, j]$ is the span, $\bullet$ marks how much has been matched, and $w$ is the log-probability weight accumulated so far. Below are the complete items (dot at the end) in each chart column.

```
Sentence: time flies like an arrow

=== Chart column 1 ===
  [0,1] N -> time *      w=0.0000
  [0,1] NP -> N *        w=-0.9163

=== Chart column 2 ===
  [0,2] NP -> N N *      w=-2.5257
  [1,2] N -> flies *     w=0.0000
  [1,2] V -> flies *     w=0.0000

=== Chart column 3 ===
  [2,3] ADV -> like *    w=0.0000
```

```
  [2,3] V -> like *       w=0.0000

=== Chart column 4 ===
  [3,4] D -> an *         w=0.0000

=== Chart column 5 ===
  [0,5] S -> NP VP *      w=-6.9486
  [1,5] VP -> V ADVP *    w=-2.5257
  [2,5] ADVP -> ADV NP * w=-1.8326
  [2,5] VP -> V NP *      w=-2.5257
  [3,5] NP -> D N *       w=-0.9163
  [4,5] N -> arrow *      w=0.0000
```

### How the Chart Was Computed

The Earley algorithm proceeds column by column, applying three operations:

- **Predictor:** For every incomplete item $[i, k]$ $A \to \alpha \bullet B\ \beta$, add $[k, k]$ $B \to \bullet\ \gamma$ for all rules $B \to \gamma$ in the grammar.
- **Scanner:** For every incomplete item $[i, k]$ $A \to \alpha \bullet w_k\ \beta$ where $w_k$ is the next word, advance the dot to produce $[i, k{+}1]$ $A \to \alpha\ w_k \bullet \beta$.
- **Completer:** For every complete item $[j, k]$ $B \to \gamma \bullet$, find all items $[i, j]$ $A \to \alpha \bullet B\ \beta$ and advance their dots.

Notable observations in the chart:

- Column 1 shows "time" being recognised as both $N$ and (via $NP \to N$) as an $NP$.
- Column 2 shows "flies" recognised as both $N$ and $V$, causing ambiguity.
- Column 3 shows "like" recognised as both $ADV$ and $V$.
- Column 5 contains the complete parse $S \to NP\ VP \bullet$ $[0, 5]$, confirming a successful parse.

## Question 2

**Compute the probability of each tree emitted by the parser using the rule probabilities provided in the grammar. (10 marks)**

The sentence is structurally ambiguous. It has two valid parse trees under this grammar. The parser finds both and computes their probabilities as the product of all rule probabilities used.

### Parse 1: "time" as subject, "flies like an arrow" as VP

```
(S -> NP VP
  (NP -> N
    (N -> time))
  (VP -> V ADVP
    (V -> flies)
    (ADVP -> ADV NP
      (ADV -> like)
      (NP -> D N
        (D -> an)
        (N -> arrow)))))
```

**Probability computation:**

$$
\begin{aligned}
P(\text{Parse 1}) &= P(S \to NP\ VP) \times P(NP \to N) \times P(N \to \text{time}) \\
&\quad \times P(VP \to V\ ADVP) \times P(V \to \text{flies}) \\
&\quad \times P(ADVP \to ADV\ NP) \times P(ADV \to \text{like}) \\
&\quad \times P(NP \to D\ N) \times P(D \to \text{an}) \times P(N \to \text{arrow}) \\
&= 1.0 \times 0.35 \times 0.4 \times 0.4 \times 0.5 \times 1.0 \times 1.0 \times 0.4 \times 1.0 \times 0.4 \\
&= \mathbf{4.48 \times 10^{-3}}
\end{aligned}
$$

$\log P(\text{Parse 1}) = -5.408132$

### Parse 2: "time flies" as compound noun subject, "like an arrow" as VP

```
(S -> NP VP
  (NP -> N N
    (N -> time)
    (N -> flies))
  (VP -> V NP
    (V -> like)
    (NP -> D N
      (D -> an)
      (N -> arrow))))
```

**Probability computation:**

$$
\begin{aligned}
P(\text{Parse 2}) &= P(S \to NP\ VP) \times P(NP \to N\ N) \times P(N \to \text{time}) \times P(N \to \text{flies}) \\
&\quad \times P(VP \to V\ NP) \times P(V \to \text{like}) \\
&\quad \times P(NP \to D\ N) \times P(D \to \text{an}) \times P(N \to \text{arrow}) \\
&= 1.0 \times 0.25 \times 0.4 \times 0.2 \times 0.6 \times 0.5 \times 0.4 \times 1.0 \times 0.4 \\
&= \mathbf{9.60 \times 10^{-4}}
\end{aligned}
$$

$\log P(\text{Parse 2}) = -6.948577$

### Summary

| Parse | log-probability | Probability |
|---|---|---|
| Parse 1: time \| flies like an arrow | $-5.4081$ | $4.48 \times 10^{-3}$ |
| Parse 2: time flies \| like an arrow | $-6.9486$ | $9.60 \times 10^{-4}$ |

Table 1: Parse probabilities for "time flies like an arrow"

Parse 1 is approximately 4.67× more probable than Parse 2.

## Question 3

**Design a grammar for to give analyses for the sentence the man shot the solider with a gun and test your parser implementation. Provide the final chart. (10 marks)**

### Designed Grammar

This sentence is PP-attachment ambiguous: the prepositional phrase "with a gun" can attach either to the VP (the gun was used for shooting) or to the NP (the soldier had the gun). The grammar is designed to capture both readings.

```
1.0   S    NP VP
0.5   NP   DT N
0.3   NP   DT N PP
0.2   NP   N
0.5   VP   V NP
0.5   VP   V NP PP
1.0   PP   P NP
0.3   DT   the
0.3   DT   a
0.4   DT   an
1.0   N    man
0.5   N    soldier
0.5   N    gun
1.0   V    shot
1.0   P    with
```

## Final Earley Chart

```
Sentence: the man shot the soldier with a gun

=== Chart column 1 ===
  [0,1] DT -> the *          w=0.0000

=== Chart column 2 ===
  [0,2] NP -> DT N *         w=-1.2040
  [1,2] N -> man *           w=0.0000

=== Chart column 3 ===
  [2,3] V -> shot *          w=0.0000

=== Chart column 4 ===
  [3,4] DT -> the *          w=0.0000

=== Chart column 5 ===
  [0,5] S -> NP VP *         w=-5.1805
  [2,5] VP -> V NP *         w=-2.5903
  [3,5] NP -> DT N *         w=-1.8971
  [4,5] N -> soldier *       w=0.0000

=== Chart column 6 ===
  [5,6] P -> with *          w=0.0000

=== Chart column 7 ===
  [6,7] DT -> a *            w=0.0000

=== Chart column 8 ===
  [0,8] S -> NP VP *         w=-7.7708
  [2,8] VP -> V NP *         w=-5.6914
  [2,8] VP -> V NP PP *      w=-5.1805
  [3,8] NP -> DT N PP *      w=-4.4874
  [5,8] PP -> P NP *         w=-2.5903
  [6,8] NP -> DT N *         w=-1.8971
  [7,8] N -> gun *           w=0.0000
```

## Parse Trees

**Parse 1: VP attachment** (prob $= 4.22 \times 10^{-4}$): "the gun was used for shooting"

```
(S -> NP VP
  (NP -> DT N
```

```
    (DT -> the)
    (N -> man))
  (VP -> V NP PP
    (V -> shot)
    (NP -> DT N
      (DT -> the)
      (N -> soldier))
    (PP -> P NP
      (P -> with)
      (NP -> DT N
        (DT -> a)
        (N -> gun)))))
```

**Parse 2: NP attachment** (prob $= 2.53 \times 10^{-4}$): "the soldier had the gun"

```
(S -> NP VP
  (NP -> DT N
    (DT -> the)
    (N -> man))
  (VP -> V NP
    (V -> shot)
    (NP -> DT N PP
      (DT -> the)
      (N -> soldier)
      (PP -> P NP
        (P -> with)
        (NP -> DT N
          (DT -> a)
          (N -> gun))))))
```

Both readings are linguistically valid. Parse 1 (VP-attach) is assigned higher probability by the grammar because $P(VP \to V\ NP\ PP) = 0.5 > P(NP \to DT\ N\ PP) = 0.3$.

# Question 4

## Correctness: Tracking Best Derivations

The parser maintains two separate data structures per chart column $i$:

- `chart[i]`: a `dict` mapping each item's key `(rule, dot, start, end)` to a single `Item` object. This is used solely for $O(1)$ duplicate detection. Once a key is inserted, it is never updated in-place.
- `all_items[i]`: a `list` containing *every* `Item` ever pushed to column $i$, including multiple items with the same key but different back-pointers (i.e., different derivations).

**Why not update in-place?** A naive approach stores one item per key and updates its back-pointer whenever a better-weight derivation arrives. This is incorrect for all-parses enumeration: the first derivation's back-pointer chain is permanently overwritten and is no longer reachable. By never updating `chart[i]` and always appending to `all_items[i]`, every derivation chain is preserved independently.

**Handling multiple derivations.** The `enum_trees()` generator recursively walks `all_items` to reconstruct trees:

```python
def enum_trees(item, all_items):
    if item.back is None:
        yield (item.rule, [])
        return
    completed, parent = item.back
    comp_cands = [it for it in all_items[completed.end]
                  if it.key() == completed.key()]
    par_cands  = [it for it in all_items[parent.end]
```

```
                    if it.key() == parent.key()]
    for comp in comp_cands:
        for par in par_cands:
            for ct in enum_trees(comp, all_items):
                for pt in enum_trees(par, all_items):
                    yield (item.rule, pt[1] + [ct])
```

For each item, all candidate items in `all_items` with the same key are found and every combination of sub-trees is explored. This correctly yields all parse trees without missing any derivation.

### Efficiency: $O(n^2)$ Space and $O(n^3)$ Time

**Space - $O(n^2)$:** The `chart` dict holds at most one entry per `(rule, dot, start, end)` tuple. Since `start` and `end` both range over $[0, n]$, and the number of distinct dotted rules is fixed by the grammar (call it $G$), the total number of chart entries is at most $G \cdot n^2 = O(n^2)$ for a fixed grammar.

**Time - $O(n^3)$:** Each unique key is pushed to the agenda exactly once (the `if key not in chart[pos]` guard ensures this). Therefore, each item is processed (popped from the agenda) at most once. The Completer, when processing a complete item ending at position $k$ that spans from $j$ to $k$, iterates over `chart[j]`, at most $O(n)$ items, and produces one new item per candidate. Summing over all complete items, the total Completer work is $O(n^3)$.

$O(1)$ push: The `_push` method performs a single dict lookup (`key not in chart[pos]`) and either a dict insert or a list append, both $O(1)$. The duplicate check is $O(1)$ because Python dicts use hash tables. Without this guard, an item could be re-added to the agenda multiple times, causing exponential reprocessing.

```
def _push(self, item, pos, chart, agenda):
    key = item.key()
    self.all_items[pos].append(item)    # always record every derivation
    if key not in chart[pos]:           # O(1) hash lookup
        chart[pos][key] = item
        agenda[pos].append(item)        # O(1) append: never re-queued
```

**Why re-queueing must be prevented:** If the same key were added to the agenda every time a new derivation arrived, the Completer would process it again, potentially triggering further completions that re-add the same items, leading to exponential blowup. The $O(1)$ duplicate check is therefore essential for maintaining polynomial complexity.

## GitHub Link

The code for this can be found at: https://github.com/garguy07/earley-parser